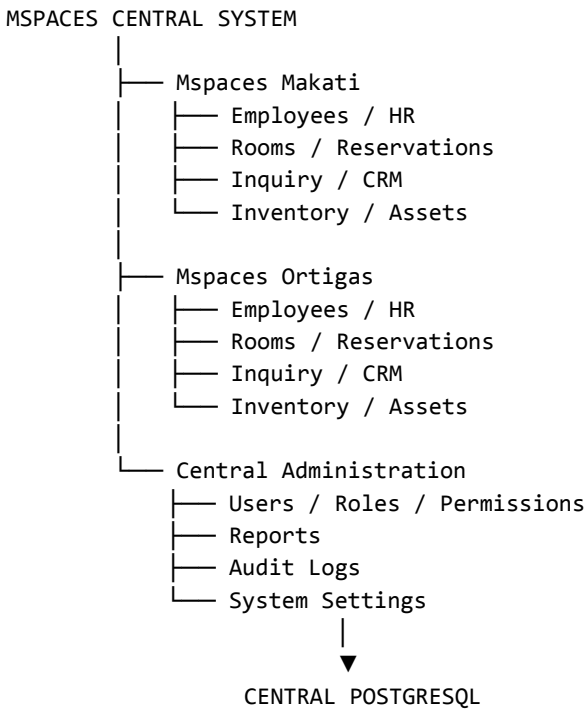
MSPACES CENTRALIZED MANAGEMENT SYSTEM

System Flow, Modules, Resources & Architecture Planning Document

Overall Structure

One centralized application serve all Mspaces branches. Branches are data records with branch-scoped access rather than separate applications.



System Modules

Authentication & User Management

- Login/logout
- Password reset
- Session management
- 2FA/MFA readiness
- Account activation/deactivation

Employee Management / 201 File

- Employee profile
- Employment information
- Emergency contacts
- Government IDs
- Contracts
- Employee documents

HR / Manager Portal

- Employee management
- Leave approvals
- Attendance

- HR reports
- Team management

Employee Portal

- My profile
- My 201 file
- Leave filing
- Leave balance
- Attendance
- Can see the payslip
- Documents
- Announcements
- Notifications

Leave Management

- Leave types
- Leave balances
- File leave
- Manager approval
- HR approval
- Attachments
- History

Room / Workspace Management

- Branches
- Buildings
- Floors
- Rooms
- Room types
- Capacity
- Rates
- Amenities
- Status

Reservations / Booking

- Availability
- Reservation creation
- Customer information
- Booking status
- Booking history

Inquiry / CRM

- Customer leads
- Inquiry status
- Follow-ups
- Assigned staff
- Notes
- Conversion tracking

Inventory Management

- Categories
- Items
- Stock in/out
- Transfers
- Adjustments
- Low-stock monitoring

Asset Management

- IT equipment
- Asset tags
- Serial numbers
- Assignment
- Warranty
- Maintenance
- Asset history

Notifications

- In-app notifications
- Email readiness
- Leave notifications
- Inquiry assignments
- Reservation notifications

Reports & Analytics

- Branch reports
- Employee reports
- Leave reports
- Inventory reports
- Room utilization
- Inquiry conversion

Audit & Security

- Audit logs
- Permission checks
- Login history
- Sensitive-data access tracking
- Security events

System Administration

- Branches
- Departments
- Positions
- Roles
- Permissions
- System settings

System Access Roles

Role	Purpose	Typical Scope
SUPER_ADMIN	Central IT / system owner	All branches and system configuration
IT_ADMIN	Technical administration	Users, roles, permissions, settings, audit logs
HR_ADMIN	HR operations	Employee records, 201 files, leave, attendance
BRANCH_MANAGER	Branch management	Assigned branch/team, approvals, rooms, inquiries
STAFF	Operational staff	Only permissions assigned to the user
EMPLOYEE	Regular employee	Own profile, leave, attendance, documents

Permission Model

```
User → Role → Permission → Module + Action + Scope
Actions: VIEW / CREATE / UPDATE / DELETE / APPROVE / REJECT / EXPORT / IMPORT
Scopes: OWN / TEAM / BRANCH / ALL
```

Branch & Multi-Branch Flow

```
branches
├── MAKATI
└── ORTIGAS
```

- branches table
- branch_id on employees, rooms, inventory and inquiries
- Branch scope on users/roles where required

Employee / 201 File Flow

```
HR / Admin
    ↓
Create Employee
    ↓
Employee Profile
    ├── Personal Information
    ├── Employment Information
    ├── Emergency Contact
    ├── Government IDs
    ├── Contract
    └── Documents
            ↓
      Access Controlled
            ↓
        Audit Logged
```

- Normalize the 201 file across related tables.
- Restrict salary, government IDs and confidential HR documents.
- Serve private documents through authorized backend endpoints, not public URLs.

Leave Management Flow

```
Employee → File Leave → Manager Review
                        ├── Reject → Rejected
                        └── Approve → HR Review
                                      ├── Reject → Rejected
                                      └── Approve → Approved → Leave Balance → Notification
```

- Store dates, leave type, reason, attachment, status and approval history.
- Make approval workflow configurable where possible.
- Audit every approval and rejection.

Room / Workspace Flow

```
Branch → Building → Floor → Room / Workspace
                              ↓
                         Availability
                              ↓
                         Reservation
                              ↓
                        Confirmation
                              ↓
                     History / Reports
```

- Suggested statuses: AVAILABLE, RESERVED, OCCUPIED, MAINTENANCE, INACTIVE.
- Separate room master data from reservation history.

Inquiry / CRM Flow

```
Customer / Lead → New Inquiry → Assigned Staff → Contacted
                  → Follow-up → Site Visit / Negotiation
                  → Converted OR Lost → Reports
```

- Track source, branch, interested space, requirements, budget, assigned staff, status, notes and follow-ups.

Inventory & Asset Flow

```
Inventory: Stock In / Stock Out / Transfer / Adjustment / Return / Damaged / Disposed
Assets: Asset Tag / Serial Number / Assignment / Warranty / Maintenance / History
```

Keep consumable inventory separate from trackable assets such as laptops, monitors, printers, routers, access points and CCTV equipment.

Database / PostgreSQL Structure

```
mspaces_db
├── AUTH: users, roles, permissions, role_permissions, user_roles
├── ORGANIZATION: branches, departments, positions
├── HR: employees, employee_documents, employee_contracts,
│       employee_emergency_contacts, employee_government_ids,
│       leave_types, leave_balances, leave_requests, leave_approvals
├── WORKSPACE: buildings, floors, rooms, room_amenities, room_reservations
├── CRM: customers, inquiries, inquiry_followups
├── INVENTORY: inventory_categories, inventory_items,
│              inventory_transactions, assets, asset_assignments, asset_history
└── SYSTEM: notifications, announcements, audit_logs, system_settings
```

Technology Stack & Resources

Area	Technology / Resource
Frontend	Next.js, TypeScript, Tailwind CSS, shadcn/ui
Data / Forms	TanStack Query, Axios, React Hook Form, Zod
Backend	NestJS, TypeScript, REST API
ORM	TypeORM
Database	PostgreSQL 18
Authentication	JWT / secure HttpOnly cookie strategy, Passport
Password Hashing	Argon2
Validation	class-validator, class-transformer
Cache / Queue Readiness	Redis
Infrastructure	Nginx, PM2,
Development	VS Code, Git, GitHub, Postman
File Storage	S3-compatible object storage or secured server storage
Email	SMTP provider

Security Features

- HTTPS in production
- Argon2 password hashing
- Secure HttpOnly authentication cookies
- Short-lived access tokens and controlled refresh-token handling
- RBAC plus branch-level authorization
- Global DTO validation
- Rate limiting for login and sensitive endpoints
- Secure CORS configuration
- Audit logs for sensitive actions
- File type/size/authorization checks
- Database backups with tested restoration
- Secrets never committed to Git
- Security headers
- Least-privilege database/application accounts

Audit Log Design

```
Audit Event
├── User
├── Action
├── Module
├── Record ID
├── Branch
├── Timestamp
├── IP Address
├── Result
├── Before Data (when appropriate)
└── After Data (when appropriate)

Examples: LOGIN, LOGOUT, CREATE, UPDATE, DELETE, APPROVE,
REJECT, EXPORT, DOWNLOAD, ROLE_CHANGE, PASSWORD_CHANGE
```

Notification Flow

```
Business Event → Notification Service
                 ├
                 ├── Email
                 └── Future: SMS / Push
                       ↓
                   Recipient
                       ↓
                 Read / Unread / History
```

Dashboard Structure

SuperAdmin Dashboard

- Total branches
- Total employees
- Room availability
- Pending leave
- Open inquiries
- Low-stock items
- System/security alerts

Branch Manager Dashboard

- Branch employees
- Attendance summary
- Pending approvals
- Room reservations
- New inquiries
- Inventory alerts

Employee Dashboard

- Attendance
- Leave balance
- Pending requests
- Announcements
- Notifications
- Personal profile

UI / Design System

- Primary theme: Red / Black / White
- Use red for primary actions and brand accents rather than every surface
- White/light-neutral application backgrounds
- Dark text and neutral borders
- Minimal cards, borders, spacing and shadows
- Status badges should include text/icons, not color alone
- Use a consistent typography, spacing, table, form, dialog and navigation system

```
Primary: Mspaces Red
Text: Black / Dark Gray
Background: White / Light Gray
Success: Green | Warning: Amber | Danger: Red
Style: Minimal, clean, business-focused, responsive, accessible
```

API Structure

```
/api/auth
/api/users
/api/roles
/api/permissions
/api/branches
/api/departments
/api/positions
/api/employees
/api/leaves
/api/attendance
/api/buildings
/api/floors
/api/rooms
/api/reservations
/api/customers
/api/inquiries
/api/inventory
/api/assets
/api/notifications
/api/audit
/api/reports
```

Development Workflow

1. Design/update database
2. Create TypeORM entity
3. Create migration
4. Run migration
5. Implement NestJS DTO/service/controller
6. Implement frontend feature
7. Add permission and branch-scope checks
8. Add audit logging
9. Test API with Postman
10. Test UI workflow

Development Phases

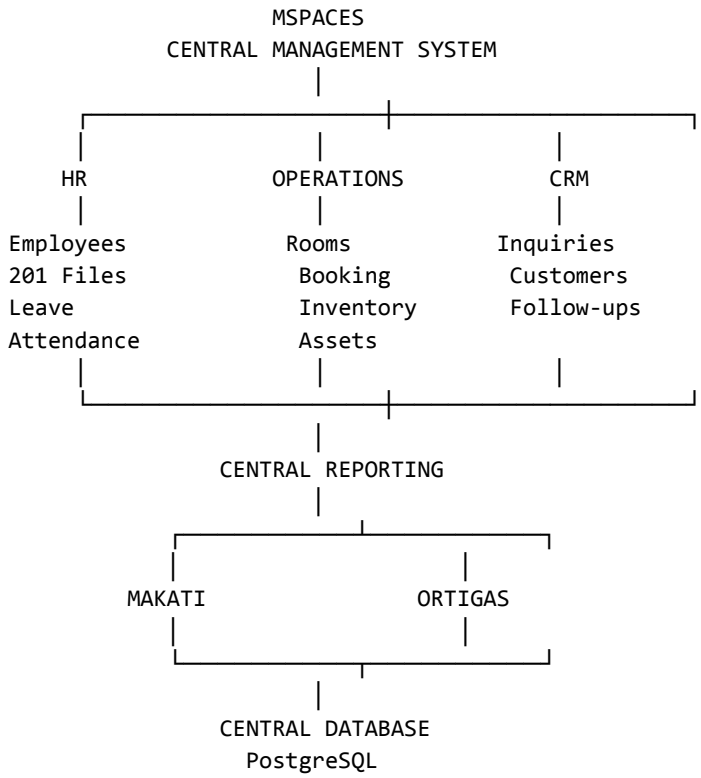
Phase	Scope
Phase 1 — Foundation	Project setup, PostgreSQL, TypeORM, migrations, authentication, users, roles, permissions, branches, audit logs
Phase 2 — HR	Employees, 201 files, leave, approvals, attendance
Phase 3 — Operations	Buildings, floors, rooms, workspace status, reservations
Phase 4 — CRM	Customers, inquiries, assignments, follow-ups, conversion tracking
Phase 5 — Inventory & IT Assets	Inventory, transfers, assets, assignments, maintenance
Phase 6 — Reporting	Branch, HR, room, inventory and inquiry reports
Phase 7 — Advanced	Customer portal, online booking, payments, mobile app, SMS/push

MVP Scope

- Authentication
- Users
- Roles
- Permissions
- Branches
- Employees
- 201 File
- Leave filing
- Manager approval
- HR approval
- Audit logs

The MVP should establish the security, database and branch-access foundation before adding every operational feature.

The Bigger Vision



Long-term vision: evolve from an internal HR/operations platform into a centralized coworking-space business management platform. New branches, employees, rooms, customers and workflows should be added through configuration and well-defined modules rather than duplicated applications.

Implementation principle: Branch is data. Role is permission. Permission determines the action. The database remains normalized. Sensitive actions are authorized and audited.

Appendix — Initial Resource Checklist

☐ PostgreSQL 18

☐ Node.js LTS

☐ VS Code

☐ Next.js

☐ NestJS

☐ TypeORM

☐ pg PostgreSQL driver

☐ Redis

☐ shadcn/ui

☐ TanStack Query

☐ React Hook Form

☐ Zod

☐ Argon2

☐ Nginx

☐ PM2

☐ Backup storage

☐ SMTP provider